

TP 1 - Introduction à MATLAB

Ali ZAINOUL
ali.zainoul.az@gmail.com

9 novembre 2025

1. Fonctions d’affichage : disp, fprintf et sprintf

MATLAB propose plusieurs fonctions pour afficher des informations dans la console. Comprendre ces fonctions et leurs options de formatage est essentiel pour produire une sortie claire et lisible.

disp

La fonction `disp` est utilisée pour afficher simplement du texte ou des variables sans mise en forme particulière :

```
disp('hello_world!');  
disp(3.1415);
```

fprintf

La fonction `fprintf` permet d’afficher une sortie formatée dans la fenêtre de commande.

```
fprintf('the_value_of_pi_is_approximately: %.2f\n', 3.1415);
```

sprintf

```
formatted_str = sprintf('pi_to_two_decimals_is: %.2f', 3.1415);  
disp(formatted_str);
```

Spécificateurs de format

2. Fonctions d’entrée utilisateur

input

```
user_age = input('enter_your_age: ');  
user_name = input('enter_your_name: ', 's');
```

Spécificateur	Type	Exemple
%d	Entier	fprintf('integer: %d', 10);
%f	Réel	fprintf('float: %.2f', 3.1415);
%s	Chaîne	fprintf('string: %s', 'matlab');

TABLE 1 – Spécificateurs de format courants

menu

```
user_choice = menu('choose_a_color', 'red', 'blue', 'green');
```

3. Scripts et fonctions

3.1 Scripts

```
% myscript.m
x = 5;
y = 3;
disp(x + y);
```

3.2 Fonctions

```
function output_val = square_number(input_val)
    output_val = input_val ^ 2;
end
```

4. Structures conditionnelles

Les structures conditionnelles permettent d'exécuter du code selon une condition logique.

4.1 Structure if-else

```
x = input('enter_a_number: ');
if mod(x,2) == 0
    disp('the_number_is_even');
else
    disp('the_number_is_odd');
end
```

4.2 Structure elseif

```
score = input('enter_score: ');
if score >= 90
    disp('grade_A');
elseif score >= 75
    disp('grade_B');
else
    disp('grade_C');
end
```

5. Boucles

Les boucles permettent de répéter une série d'instructions plusieurs fois.

5.1 Boucle for

```
for i = 1:5
    fprintf('iteration_number_%d\n', i);
end
```

5.2 Boucle while

```
count = 1;
while count <= 5
    fprintf('count_%d\n', count);
    count = count + 1;
end
```

6. Tableaux et matrices

Les tableaux sont la base de MATLAB (Matrix Laboratory).

6.1 Création

```
A = [1 2 3 4];
B = [1; 2; 3; 4];
```

6.2 Opérations de base

```
sum_A = sum(A);
mean_A = mean(A);
std_A = std(A);
```

6.3 Tableaux aléatoires

```
R = rand(1,5); % random numbers between 0 and 1
```

7. Tableaux de cellules

Les cellules permettent de stocker des éléments de types variés.

```
C = {'Alice', 25, [90 85 88]};
fprintf('name: %s\n', C{1});
```

8. Structures

Les structures regroupent des données hétérogènes dans des champs nommés.

```
student.name = 'Alice';
student.age = 21;
student.grades = [12 15 14];
student.average = mean(student.grades);
fprintf('student: %s, average: %.2f\n', student.name, student.average);
```

9. Exercices

Exercice 1 : Utiliser disp et fprintf

Écrivez un script qui affiche les éléments suivants :

- Un message **Learning MATLAB Formatting!** à l'aide de **disp**.
- La racine carrée de 10 avec deux décimales à l'aide de **fprintf**.

Exercice 2 : Créer un menu simple

Écrivez un script qui utilise la fonction **menu** pour demander à l'utilisateur de sélectionner son langage de programmation préféré (par exemple : MATLAB, Python, Java).

Exercice 3 : Écrire une fonction

Créez une fonction **celsius_to_fahrenheit** qui convertit une température de degrés Celsius en degrés Fahrenheit. La formule de conversion est :

$$F = \frac{9}{5}C + 32$$

Testez la fonction à partir d'un script avec des valeurs d'exemple.

Exercice 4 : Utiliser sprintf pour formater une chaîne

Écrivez un script qui calcule l'aire d'un cercle de rayon 5 et formate le résultat dans une chaîne à l'aide de **sprintf**.

Exercice 5 : Utiliser des boucles et des conditions

Écrivez un script qui :

- Demande à l'utilisateur d'entrer un entier positif.
- Affiche tous les nombres pairs inférieurs ou égaux à ce nombre à l'aide d'une boucle **for**.
- Utilise une structure **if** pour vérifier si le nombre est pair ou impair.

Exercice 6 : Créer un tableau et afficher ses statistiques

Écrivez un script qui :

- Crée un vecteur de nombres aléatoires à l'aide de `rand`.
- Calcule la moyenne, l'écart-type et la somme.
- Affiche ces valeurs à l'aide de `fprintf` avec un formatage clair.

Exercice 7 : Lecture d'entrées multiples

Créez un script qui demande à l'utilisateur d'entrer trois valeurs numériques, puis affiche :

- Leur somme.
- Leur moyenne.
- Le maximum et le minimum.

Exercice 8 : Créer une fonction personnalisée

Créez une fonction `stats_vector` qui prend un vecteur en entrée et retourne :

- sa longueur ;
- sa somme ;
- sa moyenne ;
- son écart-type.

Affichez les résultats dans un script principal à l'aide de `fprintf` et d'une chaîne formatée `fmt`.

Exercice 9 : Affichage de texte formaté avec `fmt`

Reprenez l'exercice précédent et modifiez le script principal pour utiliser une seule chaîne `fmt` contenant plusieurs lignes. Affichez les résultats des trois premiers vecteurs à l'aide d'une boucle et de `fprintf(fmt, ...)`.

Exercice 10 : Chaînes de caractères et tableaux de cellules

Écrivez un script qui :

- Crée un tableau de cellules contenant plusieurs prénoms.
- Parcourt chaque élément à l'aide d'une boucle `for`.
- Utilise `fprintf` pour afficher chaque prénom avec son index.

Exercice 11 : Combiner `sprintf` et `disp`

Créez un script qui :

- Utilise `sprintf` pour formater un message personnalisé.
- Stocke ce message dans une variable `msg`.
- Affiche `msg` à l'aide de `disp`.

Exercice 12 : Manipulation de structures

Écrivez un script qui :

- Crée une structure **student** avec les champs **name**, **age**, et **grades**.
- Calcule la moyenne des notes et l'ajoute à la structure sous le champ **average**.
- Affiche le contenu complet de la structure avec **fprintf** et **sprintf**.